

Multi-Cloud Disaster Recovery System

System Design Document

Detailed Component, Data, and Workflow Design

Document Type: Design

Version 1.0 | Prepared: August 2026

Industry Skills: AWS - Azure - Disaster Recovery - Automation - High Availability

1. Purpose of This Document

This document translates the architecture into detailed, implementable design: module responsibilities, data models, APIs/interfaces, state machines, and UX for the monitoring dashboard. It is intended for engineers building and testing each subsystem.

2. Module Breakdown

Module	Responsibility	Key Interfaces
Deployment Manager	Provisions identical infra in each region via IaC templates	Terraform modules, CI/CD pipeline triggers
Replication Manager	Configures and monitors DB/storage replication, exposes lag metrics	DB replication APIs, CloudWatch/Azure Monitor metrics
Backup Manager	Schedules backups, verifies integrity, manages retention	AWS Backup API / Azure Backup API, restore-test job
Health Monitor	Runs synthetic + infra health checks, aggregates health score per region	HTTP/TCP probes, CloudWatch Alarms, Azure Monitor Alerts
Failover Orchestrator	State machine that executes detect -> confirm -> promote -> redirect -> notify	Step Functions / Logic Apps, DNS API, DB promotion API
Notification Service	Sends alerts to stakeholders via multiple channels	SNS, Email/SMS/Slack/PagerDuty webhook
Dashboard / Reporting UI	Displays live region health, replication lag, DR event history	REST/GraphQL API over health + event data store

3. Health Check Design

Each region is scored using a weighted composite of independent signals so that no single false signal triggers failover.

Signal	Check Frequency	Weight	Failure Threshold
Application HTTP endpoint (synthetic transaction)	10s	35%	3 consecutive failures
Database connectivity + replication lag	15s	30%	Lag > 60s or unreachable
Load balancer / compute health (instance-level)	15s	20%	>50% unhealthy targets
Network reachability between regions	30s	15%	3 consecutive timeouts

A region is marked **DEGRADED** when composite health drops below 70%, and **DOWN** when it drops below 30% for a sustained confirmation window (default 45 seconds). Only a DOWN state can trigger automated failover.

4. Failover State Machine

The Recovery Orchestration Engine is modeled as a finite state machine to guarantee that failover steps execute in order, are idempotent, and can be resumed if interrupted.

```
[HEALTHY] --primary degraded--> [MONITORING]
[MONITORING] --confirmed down (45s)--> [FAILOVER_INITIATED]
[MONITORING] --recovers--> [HEALTHY]
[FAILOVER_INITIATED] --DB promoted--> [DB_PROMOTED]
[DB_PROMOTED] --compute scaled--> [COMPUTE_READY]
[COMPUTE_READY] --DNS/traffic switched--> [TRAFFIC_SWITCHED]
[TRAFFIC_SWITCHED] --stakeholders notified--> [FAILED_OVER]
[FAILED_OVER] --primary verified healthy--> [FAILBACK_PENDING]
[FAILBACK_PENDING] --data re-synced + traffic shifted back--> [HEALTHY]
```

5. Data Model (Core Entities)

Entity	Key Fields	Notes
Region	region_id, provider, role (primary/secondary), status, last_health_score	One row per deployed region
HealthCheckResult	check_id, region_id, signal_type, timestamp, pass/fail, latency_ms	Raw time-series health data
ReplicationStatus	resource_id, source_region, target_region, lag_seconds, last_synced_at	Tracks DB/storage replication lag
BackupJob	job_id, resource_id, schedule, status, verified (bool), retention_expiry	Backup execution and integrity records
FailoverEvent	event_id, trigger_type (auto/manual), start_time, end_time, from_region, to_region, state_log	Full audit trail of every DR event
NotificationLog	notification_id, event_id, channel, recipient, sent_at, ack_status	Tracks alert delivery

6. API / Interface Design (Representative)

Endpoint	Method	Purpose
/api/v1/regions/health	GET	Returns current composite health score and status for all regions

Endpoint	Method	Purpose
/api/v1/replication/status	GET	Returns replication lag for each replicated resource
/api/v1/failover/trigger	POST	Manually trigger failover (requires elevated role)
/api/v1/failover/abort	POST	Abort an in-progress automated failover
/api/v1/failover/events	GET	List historical failover/failback events with full timeline
/api/v1/backups/{resource_id}	GET	List backup jobs and verification status for a resource
/api/v1/backups/restore-test	POST	Trigger an automated restore drill

7. Dashboard / UX Design

The monitoring dashboard is the primary operator-facing surface and should present:

- **Region Status Panel** - side-by-side health cards for primary and secondary regions (color-coded: green/healthy, amber/degraded, red/down).
- **Replication Lag Widget** - real-time chart of DB and storage replication lag against the RPO threshold line.
- **Event Timeline** - chronological log of health transitions, failover steps, and notifications, with drill-down detail.
- **Manual Controls** - protected buttons for 'Trigger Failover', 'Abort Failover', and 'Initiate Failback', gated by role-based access.
- **Backup Status Table** - last backup time, verification status, and retention for each critical resource.
- **Alert/Notification Feed** - live feed of outbound alerts and delivery acknowledgement status.

8. Design Principles

- **Idempotency:** every orchestration step can be safely retried without causing duplicate side effects.
- **Fail-safe defaults:** ambiguous or partial health signals default to escalation/manual review rather than blind auto-failover.
- **Separation of concerns:** monitoring, decision-making, and execution are distinct modules that communicate via well-defined events/APIs.
- **Auditability:** every automated decision and action is logged with sufficient detail for post-incident review.
- **Least privilege:** orchestration engine holds only the specific IAM/RBAC permissions needed to promote DB, scale compute, and update DNS.